石头剪刀布

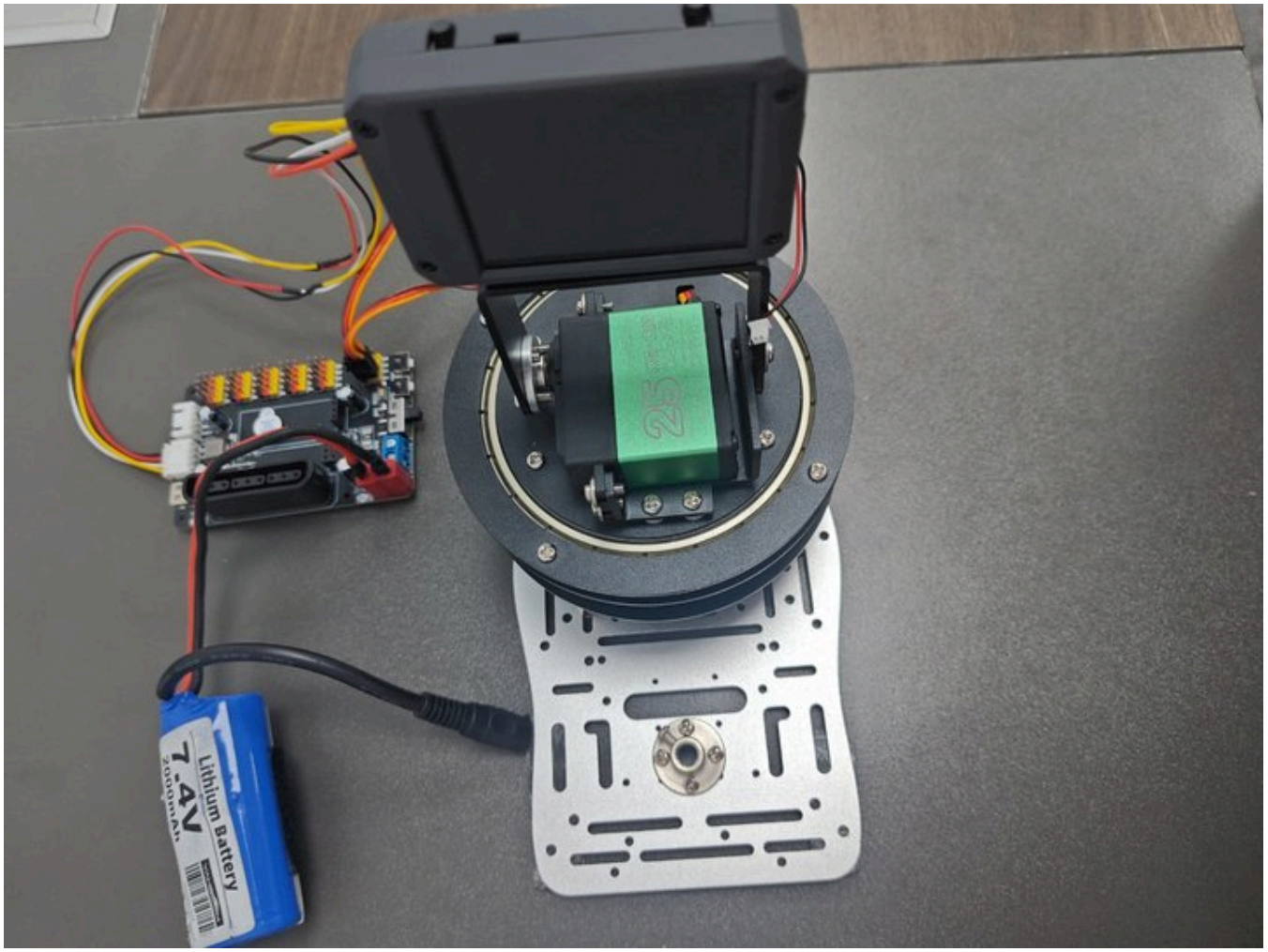
1.开篇说明

请先阅读二十四路舵机驱动板以及stm32入门教程里面的资料，了解清楚如何烧录程序、入门stm32的基础。这样方便学习和烧录后面的程序。

2.器材清单

- k230 x1
- 24路舵机驱动板 x1
- 7.4v电池 x1 购买
- 铝合金板 x1 购买

3.摆放位置

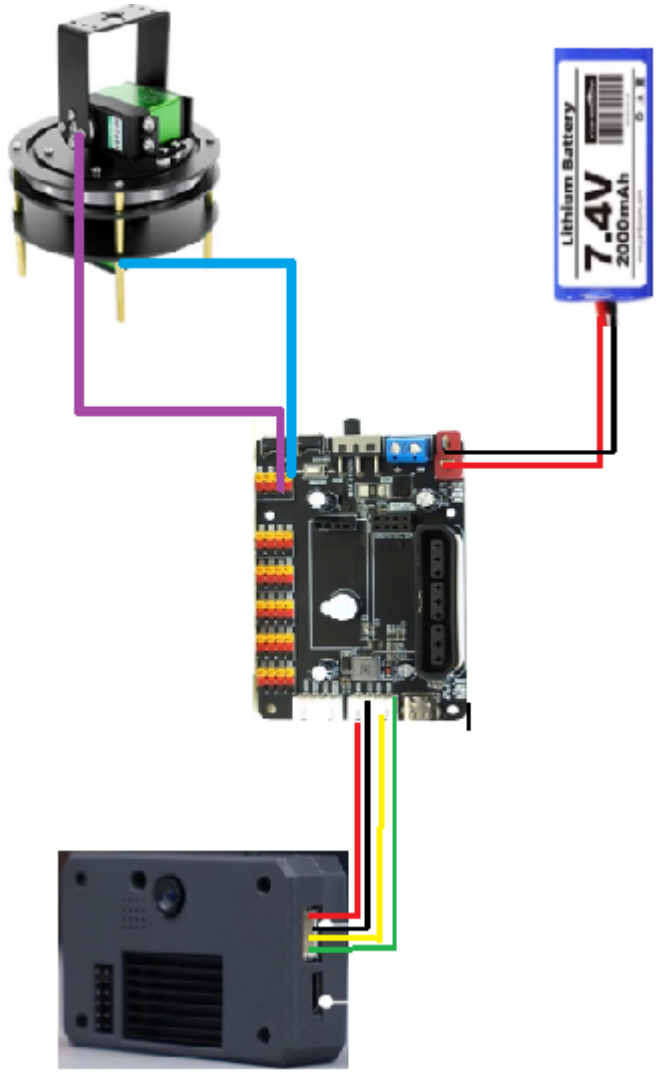


摆放位置如图所示，可以选择我们的铝基板车架作为固定k230和云台的底板，也可以自行选择能固定的底板，底盘可以胶给粘住以免云台动的幅度大导致偏移，k230固定在二维云台上面,用电池的时候如果云台转动幅度过大可以适度减小pid参数。

4.实验准备

首先按照二维云台安装教程搭建好二维云台

硬件接线：



二维云台	24路舵机驱动板
X轴舵机	S1
Y轴舵机	S2

K230视觉模块	24路电机驱动板
5V	5V
GND	GND
TX	RX3
RX	TX3

注意:插舵机的线时一定要将黄色线对准24路舵机驱动板的黄色GPIO口，褐色线对准24路舵机驱动板的黑色GPIO口。

5.K230关键代码解析

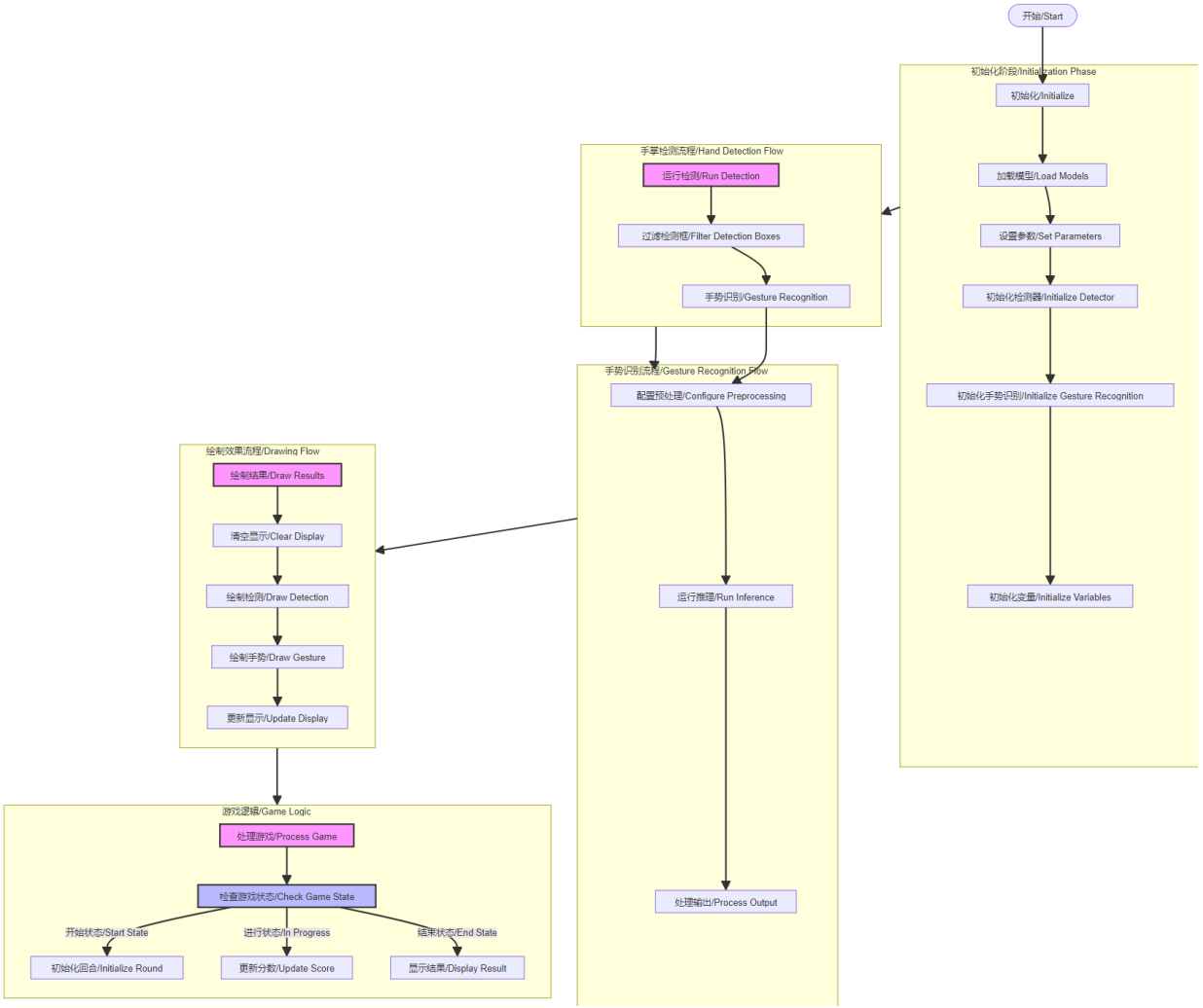
代码讲解

这是一个基于手势识别的猜拳游戏程序。主要包含以下几个部分:

1. 手掌检测类 (HandDetApp) 和手势关键点分类类 (HandKPClassApp)
    - 这两个类分别实现了手掌检测和手势关键点识别的功能,为猜拳游戏提供底层支持。
    - 它们封装了图像预处理、模型推理和后处理等操作,为上层逻辑提供统一的接口。
2. 猜拳游戏类 (FingerGuess)
    - 这个类负责整个猜拳游戏的执行逻辑。
    - 它会首先调用手掌检测和关键点分类模型,获取用户当前的手势。
    - 然后根据不同的游戏模式(必赢、必输、多局),生成对应的电脑出拳,并判断胜负。
    - 最后将游戏过程和结果绘制到屏幕上显示。
3. 游戏执行函数(exce_demo)
    - 这个函数负责初始化游戏相关的环境和资源,并进入游戏的主循环。
    - 在主循环中,它不断获取当前帧图像,调用FingerGuess类的run方法处理图像,最后显示游戏画面。

整个游戏的执行流程是:

1. 初始化游戏环境和资源
2. 进入游戏主循环
3. 在每一帧中:
    - 获取当前图像
    - 检测手掌,识别手势
    - 根据游戏逻辑生成电脑出拳,判断胜负
    - 绘制游戏画面并显示



代码详细结构

1. 初始化阶段/Initialization Phase:
    - 加载检测和识别模型/Load detection and recognition models
    - 设置各种参数/Set parameters
    - 初始化检测器和手势识别模块/Initialize detector and gesture recognition
    - 初始化游戏相关变量/Initialize game-related variables
2. 手掌检测流程/Hand Detection Flow:
    - 运行手掌检测/Run hand detection
    - 过滤检测结果/Filter detection boxes
    - 传递给手势识别模块/Pass to gesture recognition
3. 手势识别流程/Gesture Recognition Flow:
    - 配置预处理流程/Configure preprocessing
    - 执行模型推理/Run model inference
    - 处理推理输出/Process output
4. 绘制效果流程/Drawing Flow:
    - 清空显示/Clear display

- 绘制检测结果/Draw detection
- 绘制手势识别结果/Draw gesture recognition
- 更新显示/Update display

5. 游戏逻辑/Game Logic:
    - 处理游戏逻辑/Process game logic
    - 检查游戏状态/Check game state
    - 初始化新回合/Initialize new round
    - 更新玩家和AI的得分/Update player and AI scores
    - 显示最终结果/Display final result

关键代码

```
# 猜拳游戏任务类 / Rock-paper-scissors game task class
class FingerGuess:
    def __init__(self, hand_det_kmodel, hand_kp_kmodel, det_input_size, kp_input_size,
labels, anchors, confidence_threshold=0.25, nms_threshold=0.3, nms_option=False, strides=[8,
16, 32], guess_mode=3, rgb888p_size=[1280, 720], display_size=[1920, 1080], debug_mode=0):
        # 手掌检测模型路径 / Hand detection model path
        self.hand_det_kmodel = hand_det_kmodel
        # 手掌关键点模型路径 / Hand keypoint model path
        self.hand_kp_kmodel = hand_kp_kmodel
        # 手掌检测模型输入分辨率 / Hand detection model input resolution
        self.det_input_size = det_input_size
        # 手掌关键点模型输入分辨率 / Hand keypoint model input resolution
        self.kp_input_size = kp_input_size
        self.labels = labels
        # anchors / Anchor boxes
        self.anchors = anchors
        # 置信度阈值 / Confidence threshold
        self.confidence_threshold = confidence_threshold
        # nms阈值 / Non-Maximum Suppression threshold
        self.nms_threshold = nms_threshold
        # nms选项 / NMS option
        self.nms_option = nms_option
        # 特征图针对输入的下采样倍数 / Feature map downsampling multipliers relative to input
        self.strides = strides
        # sensor给到AI的图像分辨率，宽16字节对齐 / Image resolution from sensor to AI, width
aligned to 16 bytes
        self.rgb888p_size = [ALIGN_UP(rgb888p_size[0], 16), rgb888p_size[1]]
        # 视频输出VO分辨率，宽16字节对齐 / Video output VO resolution, width aligned to 16 bytes
        self.display_size = [ALIGN_UP(display_size[0], 16), display_size[1]]
        # debug_mode模式 / Debug mode
        self.debug_mode = debug_mode
        self.guess_mode = guess_mode
        # 石头剪刀布的贴图array / Rock-paper-scissors image arrays
        self.five_image = self.read_file("/sdcard/utils/five.bin")
        self.fist_image = self.read_file("/sdcard/utils/fist.bin")
        self.shear_image = self.read_file("/sdcard/utils/shear.bin")
```

```
        self.counts_guess = -1
# 猜拳次数 计数 / Count of rock-paper-scissors rounds
        self.player_win = 0
 # 玩家 赢次计数 / Player win count
        self.k230_win = 0
 # k230 赢次计数 / K230 win count
        self.sleep_end = False
# 是否 停顿 / Whether to pause
        self.set_stop_id = True
 # 是否 暂停猜拳 / Whether to suspend the game
        self.LIBRARY = ["fist", "yeah", "five"]
 # 猜拳 石头剪刀布 三种方案的dict / Dictionary of three rock-paper-scissors options
        self.hand_det = HandDetApp(self.hand_det_kmodel, self.labels,
model_input_size=self.det_input_size, anchors=self.anchors,
confidence_threshold=self.confidence_threshold, nms_threshold=self.nms_threshold,
nms_option=self.nms_option, strides=self.strides, rgb888p_size=self.rgb888p_size,
display_size=self.display_size, debug_mode=0)
        self.hand_kp = HandKPClassApp(self.hand_kp_kmodel,
model_input_size=self.kp_input_size, rgb888p_size=self.rgb888p_size,
display_size=self.display_size)
        self.hand_det.config_preprocess()
•
    # run函数 / Run function
    def run(self, input_np):
        # 先进行手掌检测 / First perform hand detection
        det_boxes = self.hand_det.run(input_np)
        boxes = []
        gesture_res = []
        for det_box in det_boxes:
            # 对检测的手做手势识别 / Perform gesture recognition on detected hands
            x1, y1, x2, y2 = det_box[2], det_box[3], det_box[4], det_box[5]
            w, h = int(x2 - x1), int(y2 - y1)
            if (h < (0.1 * self.rgb888p_size[1])):
                continue
            if (w < (0.25 * self.rgb888p_size[0]) and ((x1 < (0.03 * self.rgb888p_size[0]))
or (x2 > (0.97 * self.rgb888p_size[0])))):
                continue
            if (w < (0.15 * self.rgb888p_size[0]) and ((x1 < (0.01 * self.rgb888p_size[0]))
or (x2 > (0.99 * self.rgb888p_size[0])))):
                continue
            self.hand_kp.config_preprocess(det_box)
            results_show, gesture = self.hand_kp.run(input_np)
            boxes.append(det_box)
            gesture_res.append(gesture)
        return boxes, gesture_res
•
    # 绘制效果 / Draw results
    def draw_result(self, pl, dets, gesture_res):
        pl.osd_img.clear()
        # 手掌的手势分类得到用户的出拳，根据不同模式给出开发板的出拳，并将对应的贴图放到屏幕上显示
        # Get user's gesture from hand gesture classification, determine the board's gesture
based on different modes, and display corresponding images on screen
        if (len(dets) >= 2):
```

```
            pl.osd_img.draw_string_advanced(self.display_size[0] // 2 - 50,
self.display_size[1] // 2 - 50, 30, "请保证只有一只手入镜 Make sure only one hand on the
screen", color=(255, 255, 0, 0))
        elif (self.guess_mode == 0):
            draw_img_np = np.zeros((self.display_size[1], self.display_size[0], 4),
dtype=np.uint8)
            draw_img = image.Image(self.display_size[0], self.display_size[1],
image.ARGB8888, alloc=image.ALLOC_REF, data=draw_img_np)
            if (gesture_res[0] == "fist"):
                draw_img_np[:400, :400, :] = self.shear_image
            elif (gesture_res[0] == "five"):
                draw_img_np[:400, :400, :] = self.fist_image
            elif (gesture_res[0] == "yeah"):
                draw_img_np[:400, :400, :] = self.five_image
            pl.osd_img.copy_from(draw_img)
        elif (self.guess_mode == 1):
            draw_img_np = np.zeros((self.display_size[1], self.display_size[0], 4),
dtype=np.uint8)
            draw_img = image.Image(self.display_size[0], self.display_size[1],
image.ARGB8888, alloc=image.ALLOC_REF, data=draw_img_np)
            if (gesture_res[0] == "fist"):
                draw_img_np[:400, :400, :] = self.five_image
            elif (gesture_res[0] == "five"):
                draw_img_np[:400, :400, :] = self.shear_image
            elif (gesture_res[0] == "yeah"):
                draw_img_np[:400, :400, :] = self.fist_image
            pl.osd_img.copy_from(draw_img)
        else:
            draw_img_np = np.zeros((self.display_size[1], self.display_size[0], 4),
dtype=np.uint8)
            draw_img = image.Image(self.display_size[0], self.display_size[1],
image.ARGB8888, alloc=image.ALLOC_REF, data=draw_img_np)
            if (self.sleep_end):
                time.sleep_ms(2000)
                self.sleep_end = False
            if (len(dets) == 0):
                self.set_stop_id = True
                return
            if (self.counts_guess == -1 and gesture_res[0] != "fist" and gesture_res[0] !=
"yeah" and gesture_res[0] != "five"):
                draw_img.draw_string_advanced(self.display_size[0] // 2 - 50,
self.display_size[1] // 2 - 50, 30, "游戏开始 GAME START", color=(255, 255, 0, 0))
                time.sleep_ms(500)
            elif (self.counts_guess == self.guess_mode):
                draw_img.clear()
                if (self.k230_win > self.player_win):
                    draw_img.draw_string_advanced(self.display_size[0] // 2 - 50,
self.display_size[1] // 2 - 50, 30, "你输了 You lose", color=(255, 255, 0, 0))
                    uart.write(bytearray([0x55, 0xaa, 0xff, 0xfa]))
                elif (self.k230_win < self.player_win):
                    draw_img.draw_string_advanced(self.display_size[0] // 2 - 50,
self.display_size[1] // 2 - 50, 30, "你赢了 You win", color=(255, 255, 0, 0))
                    uart.write(bytearray([0x55, 0xaa, 0x00, 0xfa]))
```

```
                else:
                    draw_img.draw_string_advanced(self.display_size[0] // 2 - 50,
self.display_size[1] // 2 - 50, 30, "平局 | tie", color=(255, 255, 0, 0))
                self.counts_guess = -1
                self.player_win = 0
                self.k230_win = 0
                self.sleep_end = True
            else:
                if (self.set_stop_id):
                    if (self.counts_guess == -1 and (gesture_res[0] == "fist" or
gesture_res[0] == "yeah" or gesture_res[0] == "five")):
                        self.counts_guess = 0
                    if (self.counts_guess != -1 and (gesture_res[0] == "fist" or
gesture_res[0] == "yeah" or gesture_res[0] == "five")):
                        k230_guess = randint(1, 10000) % 3
                        if (gesture_res[0] == "fist" and self.LIBRARY[k230_guess] ==
"yeah"):
                            self.player_win += 1
                        elif (gesture_res[0] == "fist" and self.LIBRARY[k230_guess] ==
"five"):
                            self.k230_win += 1
                        if (gesture_res[0] == "yeah" and self.LIBRARY[k230_guess] ==
"fist"):
                            self.k230_win += 1
                        elif (gesture_res[0] == "yeah" and self.LIBRARY[k230_guess] ==
"five"):
                            self.player_win += 1
                        if (gesture_res[0] == "five" and self.LIBRARY[k230_guess] ==
"fist"):
                            self.player_win += 1
                        elif (gesture_res[0] == "five" and self.LIBRARY[k230_guess] ==
"yeah"):
                            self.k230_win += 1
                        if (self.LIBRARY[k230_guess] == "fist"):
                            draw_img_np[:400, :400, :] = self.fist_image
                        elif (self.LIBRARY[k230_guess] == "five"):
                            draw_img_np[:400, :400, :] = self.five_image
                        elif (self.LIBRARY[k230_guess] == "yeah"):
                            draw_img_np[:400, :400, :] = self.shear_image
                        self.counts_guess += 1
                        draw_img.draw_string_advanced(self.display_size[0] // 2 - 50,
self.display_size[1] // 2 - 50, 30, "第" + str(self.counts_guess) + "回合 | Round " +
 str(self.counts_guess), color=(255, 255, 0, 0))
                        self.set_stop_id = False
                        self.sleep_end = True
                    else:
                        draw_img.draw_string_advanced(self.display_size[0] // 2 - 50,
self.display_size[1] // 2 - 50, 30, "第" + str(self.counts_guess + 1) + "回合 | Round " +
 str(self.counts_guess), color=(255, 255, 0, 0))
            pl.osd_img.copy_from(draw_img)
•
    # 读取石头剪刀布的bin文件方法 / Method to read rock-paper-scissors bin files
    def read_file(self, file_name):
```

```
        image_arr = np.fromfile(file_name, dtype=np.uint8)
        image_arr = image_arr.reshape((400, 400, 4))
        return image_arr
```

6.stm32关键代码解析

- **control.c**

```
uint8_t get_point2_p()   //处理坐标函数
{
    uint8_t lable, load_1, load_2 ,load_3 ,ret=0;
    if(DataDecode1(&Uart3_RingBuff,&lable, &load_1, &load_2,&load_3) == 0){
     if(lable == 0x00){
            rec_step_p[0][0] = load_1 + load_2 + load_3 ;
//          ret=1;

        } else if(lable == 0xff){

            rec_step_p[0][1] = load_1 + load_2 + load_3 ;
//          ret=1;

        }

    return ret;
}
}
```

此代码是将k230传过的3帧数据加在一起，由于前面说过将k230的坐标数据拆成3帧数据然后分别发送，这里正是将数据加在一起这样形成正确的坐标，将坐标赋值给二维数组。

```
    void mode_1pro()
{

    USART_Cmd(USART3, ENABLE);
        get_active();

if(mode_flag==0&&rec_step_p[0][1]==1){
    go_to(0, 0, 200, 0);
    go_to(200, 0, 0, 0);
    go_to_point(0, 50, 200, 50);
    go_to_point(200, 50, 0, 50);
    USART_Cmd(USART3, DISABLE);
        rec_step_p[0][0]=0;

            mode_flag+=1;

}
```

```
else if(mode_flag==0&&rec_step_p[0][0]==1){
            USART_Cmd(USART3, DISABLE);
    go_to(0, 0, 0, 240);
    go_to(0, 240, 0, 0);
    go_to_point(0, 0, 0, 240);
    go_to_point(0, 240, 0, 0);
            mode_flag+=1;
            rec_step_p[0][1]=0;

}

}
```

此代码是在上面获得石头剪刀布结果之后，根据结果做出运动。

```
void go_to_point(uint16_t now_x, uint16_t now_y, uint16_t targ_x, uint16_t targ_y)
{
    uint16_t i;
    float step_num = 200;/* 1000步*/
    float step_x, step_y; /* 步进值  */
    float going_x = now_x, going_y = now_y; /* 当前目标值*/

    step_x = (targ_x - now_x) / step_num;
    step_y = (targ_y - now_y) / step_num;

    for(i = 0; i < step_num; i++)
{

    kfp_x.source = step_x;
    kfp_y.source = step_y;
    kfp_x.out = KalmanFilter(&kfp_x, kfp_x.source);
    kfp_y.out = KalmanFilter(&kfp_y, kfp_y.source);
    going_x += kfp_x.out;
        going_y += kfp_y.out;

        set_target(going_x, going_y);
    }
}
```

此代码是将得到的两点坐标进行细分，从而是将矩形的2个点分成许多个点，然后许多点之间做pid运算，使得舵机运动更加精准。

```
void go_to(uint16_t now_x, uint16_t now_y, uint16_t targ_x, uint16_t targ_y)
{
    uint16_t i;
    float step_num = 100;/* 250²½ */
    float step_x, step_y; /* ²½½øÖµ  */
    float going_x = now_x, going_y = now_y; /* µ±Ç°Ä¿±êÖµ*/

    step_x = (targ_x - now_x) / step_num;
```

```
    step_y = (targ_y - now_y) / step_num;

    for(i = 0; i < step_num;i++){


            kfp_x.source = step_x;
            kfp_y.source = step_y;
            kfp_x.out = KalmanFilter(&kfp_x, kfp_x.source);
            kfp_y.out = KalmanFilter(&kfp_y, kfp_y.source);


            going_x += kfp_x.out;
      going_y += kfp_y.out;

      set_target(-going_x, -going_y);

    }
}
```

此代码是将得到的两点坐标进行细分，从而是将矩形的2个点分成许多个点，然后许多点之间做pid运算，使得舵机运动更加精准。

- **bsp_servo.c**

```
ServTypdef Xserv = {
    .pwm = 1600,
    .MAXPWM = 2400,
    .MINPWM = 500,
    .MIDPWM = 1400
};

ServTypdef Yserv = {
    .pwm = 1600,
    .MAXPWM = 2500,
    .MINPWM = 300,
    .MIDPWM = 1400
};
```

此代码是设置舵机启动的起始位置，所以需要修改这个来拟合云台运动的顶点，我将云台开机初始位置设置在中间点，通过这个改变舵机的起始位置。

```
void servo_ctr(ServTypdef *servo, int PidInput)
{
    uint16_t CompSet = servo->MIDPWM + PidInput;

    if( CompSet > servo->MAXPWM ){
        CompSet = servo->MAXPWM;
    } else if ( CompSet < servo->MINPWM  ){
        CompSet = servo->MINPWM;
    }
```

```
    if(servo->pwm == CompSet){/*避免重复更改比较值*/

        return;
    }

    if( servo == &Xserv ){


    GPIO_SetBits(Servo_J1_PORT, Servo_J1_PIN );      //将舵机电平置高
    delay_us(CompSet);                    //延时脉冲值的微秒数

    GPIO_ResetBits(Servo_J1_PORT, Servo_J1_PIN );            //将舵机电平置低
    delay_ms(20 - CompSet/1000);
    }
    else if( servo == &Yserv ){


    GPIO_SetBits(Servo_J2_PORT, Servo_J2_PIN );      //将舵机电平置高
    delay_us(CompSet);                        //延时脉冲值的微秒数

    GPIO_ResetBits(Servo_J2_PORT, Servo_J2_PIN );            //将舵机电平置低
    delay_ms(20 - CompSet/1000);
    }
}
```

此代码是将pid的输出给pwm，是将pid与pwm连接的地方，同时给pwm进行限幅，也是舵机运动的核心代码，通过延时和pwm的改变来控制舵机转动，将舵机运动和pwm联系一起。

- **pid.c**

```
float pid_calculate(PID_TypeDef *PID,float CurrentValue)
{
  PID->Err =  PID->Target - CurrentValue;
    PID->Integral += PID->Err;
    if(PID->Integral > 7000){
        PID->Integral = 7000;
    }
    if(PID->Integral < -7000){
        PID->Integral = -7000;
    }    //积分限幅
    PID->PID_out = PID->Kp * PID->Err                                    /*比例*/
         + PID->Ki * PID->Integral                                  /*积分*/
            + PID->Kd * (PID->Err - PID->LastErr);                      /*微分*/

    PID->LastErr = PID->Err;
        return PID->PID_out;
}
```

此代码是位置式pid运算的配置，包括积分限幅，位置式PID是通过计算当前偏差值（目标值与实际值之差）、积分值（过去误差的总和）和微分值（误差变化率），然后将这三个部分加权相加得到PID输出，如果想微调代码想达到更好的效果可以微调主函数的pid参数。

- **bsp_usart.c**

```
uint8_t DataDecode1(RingBuff_t *ringbuff, uint8_t *data1, uint8_t *data2, uint8_t
*data3,uint8_t *data4)
{
    static uint8_t uart_dec_count;
    static uint8_t uart_rec_data[6];
    uint8_t ret = 1;

    if(Read_RingBuff(ringbuff, &uart_rec_data[uart_dec_count]) == RINGBUFF_ERR){
        return 1;

    }
        if((uart_dec_count == 0)&&(uart_rec_data[uart_dec_count] != 0x55)) {            //帧
头0x55
        uart_rec_data[uart_dec_count] = 0;
    }
        else if((uart_dec_count == 1)&&(uart_rec_data[uart_dec_count] != 0xaa)){      //第二
帧0xaa
        uart_rec_data[uart_dec_count] = 0;
        uart_rec_data[uart_dec_count-1] = 0;
        uart_dec_count = 0;
        }else if((uart_dec_count == 6)&&(uart_rec_data[uart_dec_count] != 0xfa)){      //帧
尾0xfa
        uart_rec_data[uart_dec_count] = 0;
        uart_rec_data[uart_dec_count-1] = 0;
        uart_rec_data[uart_dec_count-2] = 0;
        uart_rec_data[uart_dec_count-3] = 0;
        uart_rec_data[uart_dec_count-4] = 0;
        uart_rec_data[uart_dec_count-5] = 0;
        uart_dec_count = 0;
    }
    else{
        if(uart_dec_count == 6)//成功接收一帧数据
        {
            *data1 = uart_rec_data[2];
        *data2 = uart_rec_data[3];
                *data3 = uart_rec_data[4];
            *data4 = uart_rec_data[5];
        ret = 0;

        }
        uart_dec_count++;
        if(uart_dec_count == 7)
        {
            uart_dec_count = 0;
        }

    }
    return ret;

}
```

此代码是接收k230数据的代码，在识别到帧头为0x55，第二个数据为0xaa，帧尾为0xfa的时候才接收数据，通过三个数据定位一帧数据，由于从k230传输过来的4个顶点坐标每个顶点又通过拆分成3组数据传输过来，所以传输过来的数据是非常庞大的，为了确保传过来的数据是准确的，所以采用三个数据来确认一帧数据。

7.效果描述

k230运行石头剪刀布的程序，当你与k230进行石头剪刀布时3局2胜，当你赢了的时候k230发送信息让云台上下运动，表示点头以及对你的认可，当你输了的时候k230发送信息让云台左右运动，表示摇头以及对你的轻视。